

Application development

1. Backend Dev
2. Frontend Dev

Full stack development = backend + frontend development

=> Backend contains the business logic of our application (api class, validations, email, db communication)

=> Frontend contains user interface of the application (presentation logic)

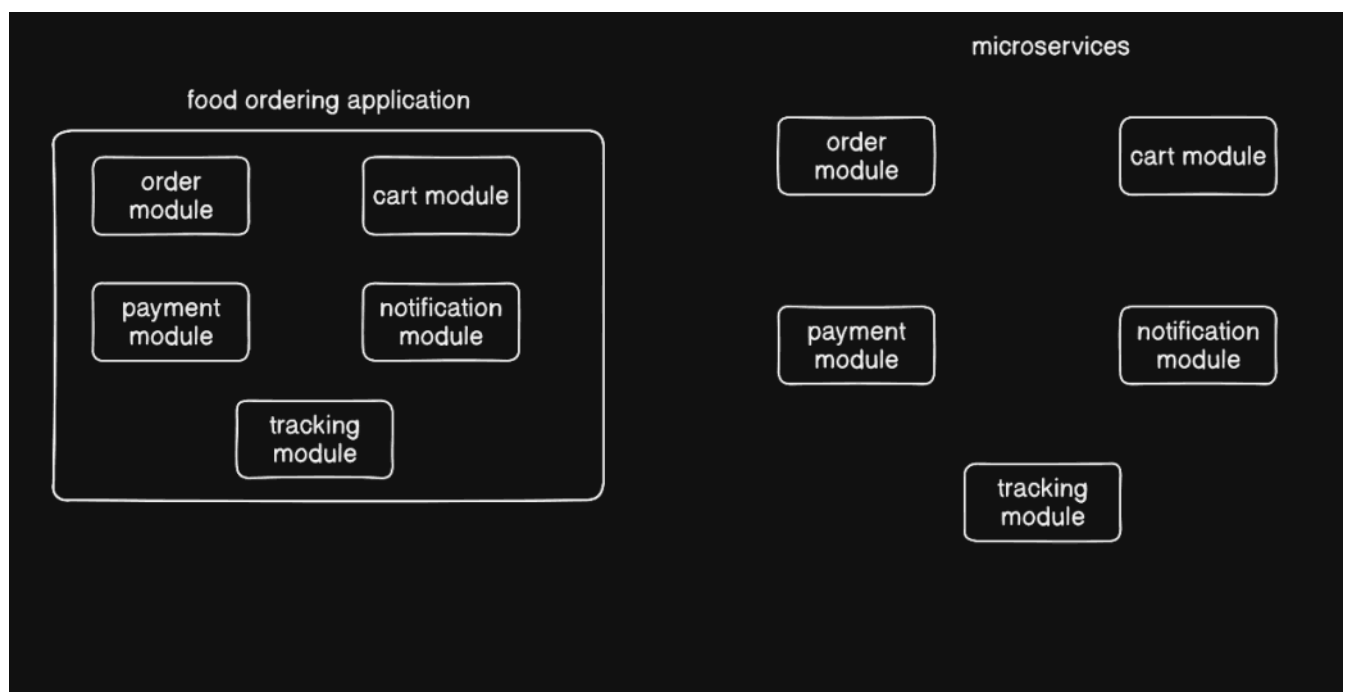
=> end user interact with frontend of the application. Frontend will interact with backend of the application

=> full stack developer are having lot of demand in the market

Monolith and Microservices

=> if we develop all the functionality in single project then it is called as Monolith Architecture based application

=> in microservices architecture, functionality will be divided into several service.



Spring framework:

Programming language: used by human to communicate with machines

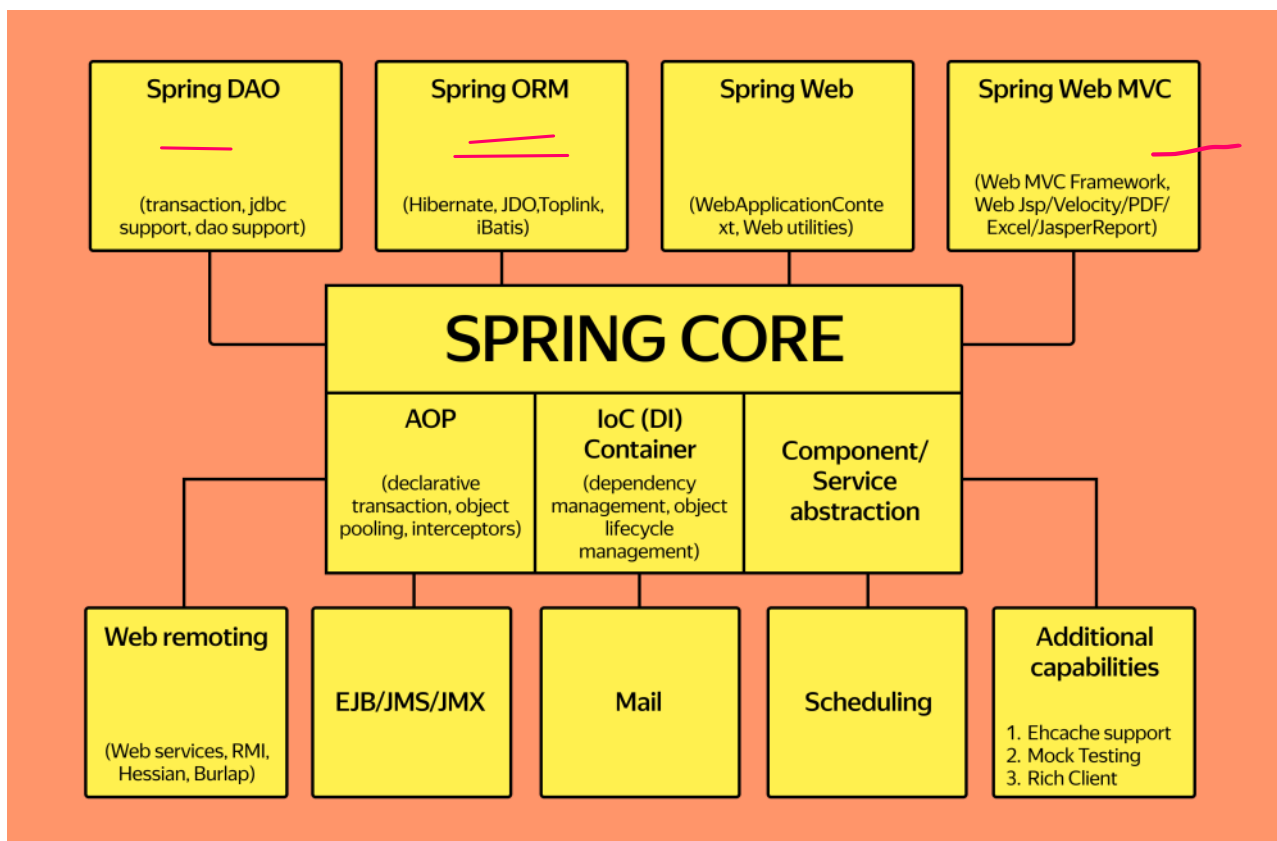
Framework: semi developed software

It provides some common logic which are required for application development
Ex: structs, spring, hibernate ...etc

If we use framework in our application development then we only need to focus on business logic.

Spring Advantage:

1. It is free & open source framework
2. Spring is very light weight framework
3. Spring is versatile framework (Spring can be integrated with any other java framework available in the market)
4. Spring framework is loosely coupled



Spring modules

1. Spring core
2. Spring context
3. Spring DAO / Spring JDBC
4. Spring AOP
5. Spring ORM
6. Spring Web MVC
7. Spring Security
8. Spring REST
9. Spring Data
10. Spring cloud

11. Spring Batch

12. Spring AI

Module	Simple meaning	When/why do we use it ?	Example requirement
Spring core	Foundation of spring	To let spring create and connect Java object instead of doing new everywhere	Orderservice need paymentservice then spring core will inject orderservice object in paymnet service
Spring context	Managing spring application/container	To manage beans, config, profile, event	@Service @Repository @Component
Spring JDBC/DAO	Easier database access using JDBC	When we want to SQL database operation without writing lots of JDBC boilerplate code	JdbcTemplate.query(.....)
Spring AOP	Run common logic around methods	Logging ,auditing ,transaction etc	Log every service method execution without adding logging code everywhere
Spring ORM	Connect spring with ORM frameworks	When using hibernate/JPA for database application	Saving student java object directly into db
Spring web mvc	Build web application	Controller, http request	/user request goes to UserController
Spring Security	Authentication + Auth	Login, role permission	Only ADMIN access
Spring REST	Build REST APIs	Create API's returning JSON over http	GET /products/10 return product json
Spring Data	Makes database code eaiser	Avoid writing repetitive CRUD queries	findByEmail(email)
Spring Cloud	Microservices	When we want to centralized configuration in microservices	20 microservices share centralized configuration
Spring Batch	Process huge amount of data in jobs	Bulk processing	Read 10 lakh record from CSV --> process--->store db
Spring AI	Add AI/LLM features	ChatGPT/Gemini RAG, emeddings, Vector DB etc	Build an AI chatbot over company document

Spring Core:

=> Spring core is base module in spring framework

=> All the other module of spring are developer on top of spring core module

=> Spring core providing fundamentals concepts of spring framework (IOC container & dependency injection)

=> Spring core is all about Managing dependencies among the classes (creating objects & injecting object)

Why do we need Spring core ?

1. Before Spring : How do we normally build java app?

Suppose we are creating an application that sent notification

That support

1. Email notification
2. SMS notification
3. WhatsApp notification

Initially as a beginner may solve this using inheritance.

Approach 1: Inheritance

```
public class Notification {
    public void send()
    {
        System.out.println("Sending notification...");
    }
}

public class WhatsAppNotification extends Notification{

    @Override
    public void send() {
        System.out.println("Sending WhatsApp Msg");
    }
}

public class Main {
    public static void main(String[] args) {

        EmailNotification notification= new EmailNotification();
        notification.send();
    }
}
```

Output: sending email

Note: inheritance work here perfectly, but inheritance represents an IS-A relationship

EmailNotification IS-A Notification

SMSNotification IS-A Notification

The problem start when we try to reuse behaviour in larger application

Problem : Inheritance explosion

Note: so developers learned an important principle: **Favor Composition over Inheritance**

Inheritance means: IS-A

Composition means : HAS-A

Composition:

```
public class Car {
    private Engine engine= new Engine();

    public void startCar()
    {
```

```

        engine.start();
        System.out.println("Car Started");
    }
}

```

Problem is still there:

Who is creating the engine ? Car class

That means Car is responsible for: Using engine + creating Engine

Then this Car class become tightly coupled with Engine class

Solution- program to an interface

package solution;

```

public class Car {
    private Engine engine;

    public Car(Engine engine)
    {
        this.engine=engine;
    }

    public void drive()
    {
        engine.start();
        System.out.println("Car is moving");
    }
}

```

Car does not create the engine

Why we use spring farmwork ?

Code--> tight coupled

Class A is dependent on Class B

1. Is-a relation
2. Has-a relation

Solution : interface --> loose coupling

Why coupling ? :

Spring core: It is base module in the spring framework.

Spring core module provide fundamentals concepts of spring farmwork

1. IOC container (Inversion of control)
2. Dependency Injection
3. Bean life cycle
4. Bean Scope
5. Autowiring

Spring core : it is all about managing **dependencies** among the classes with loosely coupling

In project we will develop several classes. All those classes we can categorize into 3 types.

1. POJO
2. Java Bean
3. Component

What is pojo ? (Plain Old Java Object)

=> Any java class which can be compiled by using JDK s/w is called as POJO class.

Ex-1 : Below class is valid POJO

Class Dummy

```
{
    int id;
    String name;
    double Sal ;
}
```

Ex 2:

Class Dummy extend Thread

```
{
    int id;
    String name;
}
```

Class Dummy extend Runnable

```
{
    //run()
}
```

Below class is not POJO servlet is part of JEE

Class Dummy Implements HttpServlet

```
{  
    //imp  
}
```

What is Java Bean ?

Any java class which follow bean specification rules is called as Java Bean.

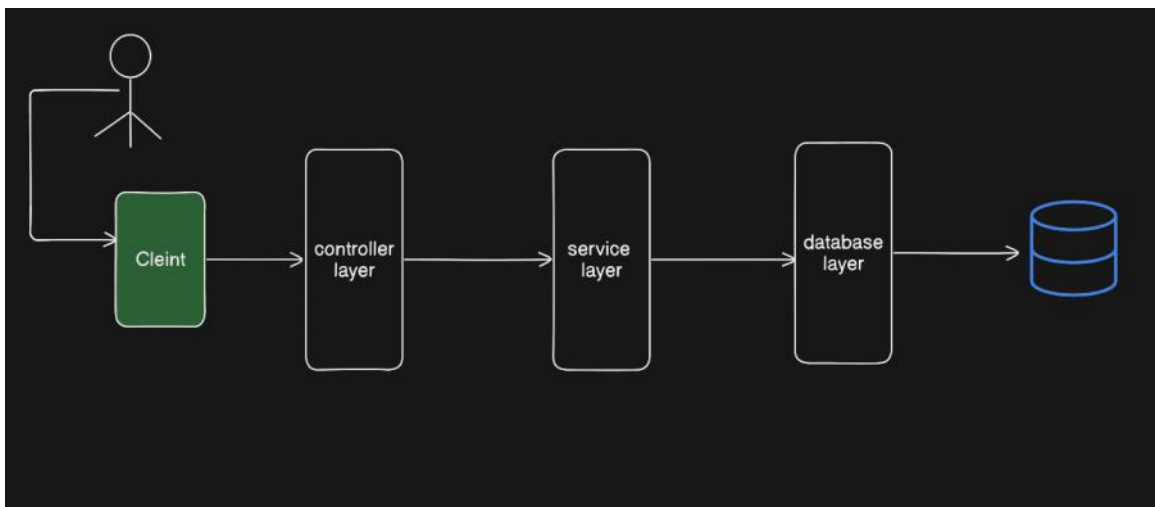
1. Class should implement serializable interface
2. Class should have private data member (variables)
3. Every private variable should have public setter & getter
4. Class should have zero-param constructor

Note: Bean classes are used to write business logic and to store retrieve data

What is Component ?

The java class which contains business logic is called Component classes

Ex: Controller, Service, Dao classes

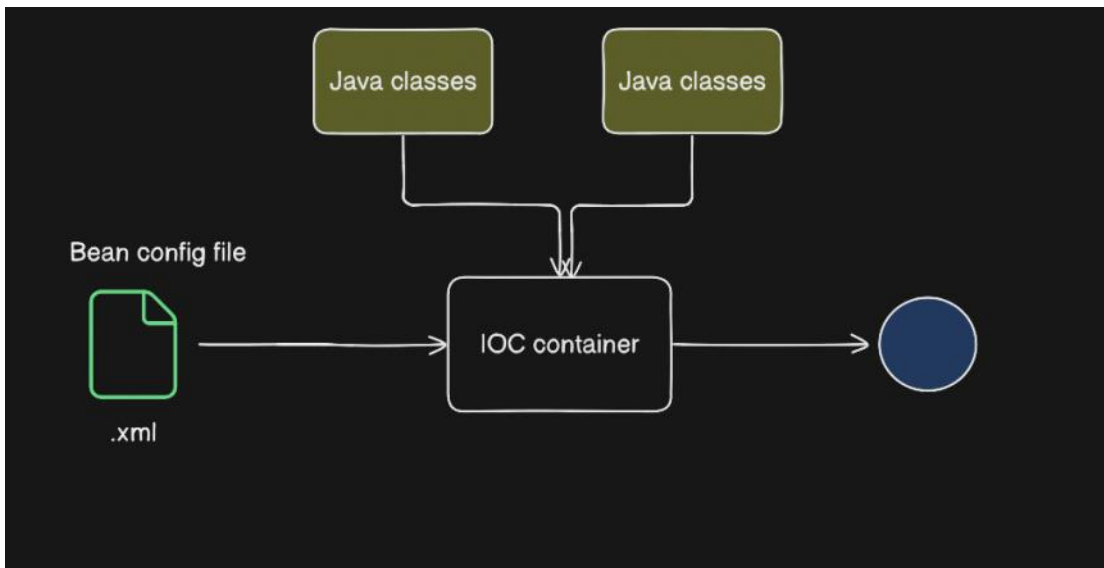


IOC Container

1. IOC stands for Inversion of control
2. IOC is responsible for Dependency Injection in Spring Application
3. Dependency Injection means creating and injecting dependent object into target class

Note: IOC container will manage life cycle of Spring beans

Note: we need to provide "java classes + Bean configuration" as input for IOC then IOC will perform DI and provide spring beans which are ready to use.



What is Spring Bean ?

Any Java Class whose lifecycle (creation and destruction) is managed by IOC is called as Spring Bean.

We can represent java class as Spring Bean in 2 ways.

1. XML Approach
2. Annotation approach

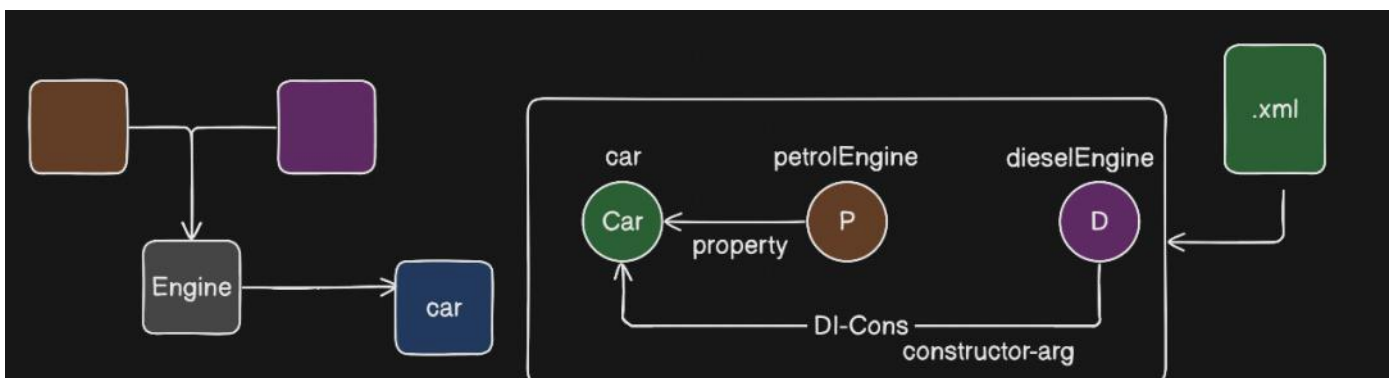
Note: In Spring we can use both xml & annotation. Spring boot will support only annotations approach.

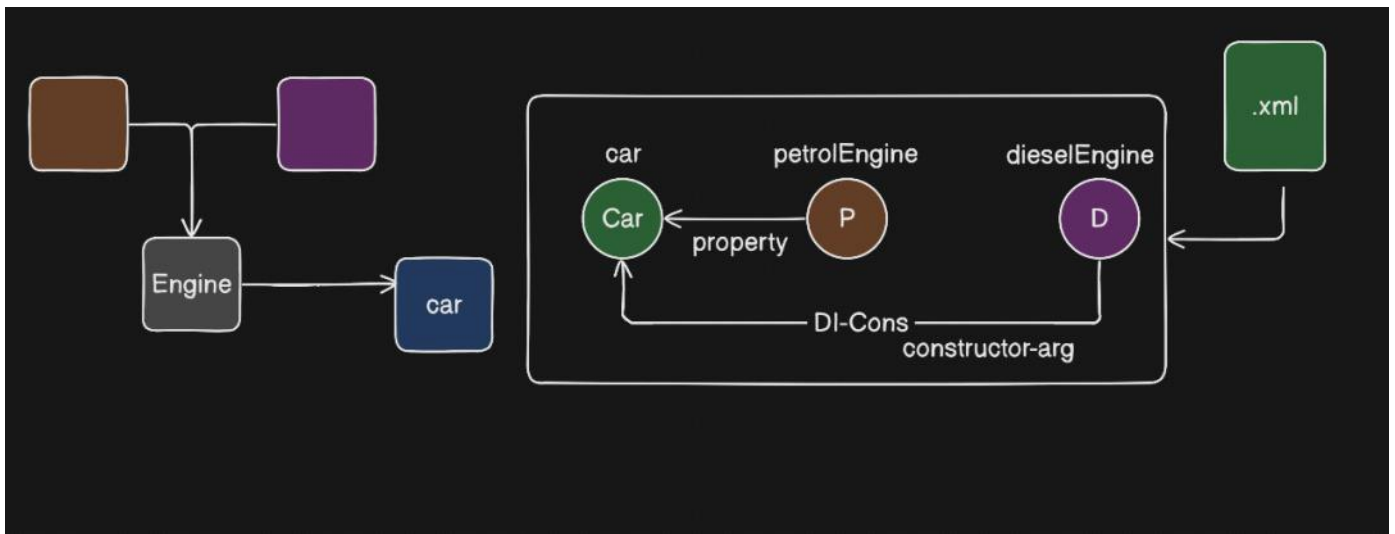
How to start IOC in spring ?

1. **BeanFactory** (outdated)
2. **ApplicationContext** (recommended)

First Application development using spring core module

1. Create maven project
2. Add "spring context" dependency from maven
3. Create required java classes
4. Create bean config and give bean definitions





Difference Between BeanFactory & ApplicationContext

BeanFactory Will follow lazy loading concept that means when we request then only It will create Bean object

ApplicationContext will follow eager loading for singleton beans. For prototype beans it will also follow lazy loading.

Note: Spring beans default scope is Singleton

Bean Scopes

Scope represent how many object should be created for a spring bean
IN spring framework we have below scope

1. Singleton
2. Prototype
3. Request
4. Session
5. Application
6. Web socket

To represent bean scope we will use "scope" attribute

```
<bean id="petrolEngine" class="org.example.PetrolEngine" scope="singleton"/>
```

Singleton scope means only one object will be created for the class in IOC container. This is default scope of spring bean.

Prototype scope mean every time new object will be created.

Note: request & session scope are related to web Application, Application scope is related to complete application

Why Spring Bean is by default Singleton ?

To save memory of JVM team made singleton as default scope for the spring beans.

Ex: TicketGenerator class is used to generate new ticket for every customer

TicketGenerator ---> Singleton bean

Ticket ----> prototype Bean

Autowiring

We can inject dependent bean into target in 2 ways

1. Manual wiring
2. Autowiring

Autowiring means IOC container will identify dependent bean and will inject into target bean (we no need to use ref attribute in bean definition)

Autowiring will work based on below modes

1. ByName
2. ByType
3. Constructor
4. No

Note: Autowiring will not work by default, we have to enable Autowiring on target bean like below.

```
<bean id="car" class="org.example.Car" auto-wire=""/>
```

byName

byName means IoC will identify dependent bean object on bean id or bean name.

byType

byType mean IoC will identify dependent bean object based on data type of the variable in target.

Private Engine eng ---> data type of eng is Engine which is class/interface

If one interface having 3 implementation then there is chance of getting ambiguity problem. To overcome This we need to use 'autowire-candidate' attribute

autowire-candidate = false --> not eligible for Autowiring

autowire-candidate = true --> eligible for autowiring

constructor

It is used to perform Autowiring by calling target class constructor

Note:

Autowiring mode	Method of injecting
byName	Setter injection
byType	Setter Injection
Constructor	Constructor injection

Home-work: Bean life cycle.

Questions:

1. What is spring bean ?
2. How to represent java class as Bean ?
3. What is IOC container ?
4. What is Bean configuration ?
5. How to start IOC container ?
6. What is dependency Injection (SI, CI & FI)
7. BeanFactory & ApplicationContext
8. Lazy Loading Vs Eager Loading
9. Bean Scope
10. Autowiring & Modes
11. Bean Life cycle methods